

Documentación del Proyecto - Preprocesamiento Ciencia de Datos

Autor: Lesly Carrasco

Curso: Cultura Digital y Sociedad - Tercer Semestre

Institución: Universidad Nacional de Chimborazo

Fecha: 11 de Mayo de 2026

1. Introducción

1.1 Objetivo del Proyecto

Implementar técnicas básicas de preprocesamiento de datos utilizando Python, Pandas y Scikit-learn, aplicando control de versiones con Git y GitHub para la gestión colaborativa del proyecto. El objetivo principal es limpiar, transformar y preparar un conjunto de datos para su posterior análisis o aplicación de algoritmos de machine learning.

1.2 Funcionalidades Implementadas

El script de preprocesamiento incluye las siguientes funcionalidades:

- **Eliminación de datos duplicados:** Remoción de filas repetidas en el dataset
- **Manejo de valores nulos:** Imputación de datos faltantes utilizando la media de las columnas numéricas
- **Conversión de variables categóricas:** Transformación de texto a formato numérico mediante One-Hot Encoding (get_dummies)
- **Normalización de datos:** Escalado de variables numéricas al rango [0, 1] usando MinMaxScaler
- **Generación de dataset procesado:** Exportación del resultado final a archivo CSV

2. Comandos Git Utilizados

2.1 Configuración Inicial del Repositorio

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/Less0miu/preprocesamiento-ciencia-
datos.git
git push -u origin main`
```

Propósito: Inicializar el repositorio **local**, realizar el primer commit y conectarlo con el repositorio remoto en GitHub.

2.2 Creación y Gestión de Ramas

```
```bash
git checkout -b feature-preprocesamiento
```

### Propósito

Crear una rama de desarrollo llamada **feature-preprocesamiento** para trabajar en nuevas funcionalidades sin afectar la rama principal **main**.

## 2.3 Agregar y Confirmar Cambios

```
git add .
git commit -m "Ejecución exitosa del preprocesamiento de datos"
```

### Propósito

- **git add .** → Añade todos los archivos modificados al área de staging.
- **git commit** → Crea un snapshot del proyecto con un mensaje descriptivo.

## 2.4 Subir Cambios al Repositorio Remoto

```
git push origin feature-preprocesamiento
```

### Propósito

Subir la rama local **feature-preprocesamiento** al repositorio remoto en GitHub para su revisión y posterior fusión.

## 2.5 Actualizar Rama Main

```
git checkout main
git pull origin main
```

### Propósito

Cambiar a la rama principal y sincronizarla con los cambios fusionados desde el repositorio remoto.

## 2.6 Eliminar Rama Después del Merge

```
git branch -d feature-preprocesamiento
```

### Propósito

Eliminar la rama de desarrollo local una vez que ha sido fusionada exitosamente con `main`, manteniendo el repositorio limpio.

## 3. Automatización con GitHub Actions

---

### 3.1 Workflow Configurado

El archivo:

```
.github/workflows/python-app.yml
```

implementa un sistema de Integración Continua (CI) utilizando GitHub Actions.

Este workflow automatiza las siguientes tareas:

- Clonado automático del repositorio
- Configuración del entorno de Python
- Instalación automática de dependencias
- Ejecución del script `preprocesamiento.py`
- Verificación del correcto funcionamiento del proyecto

El workflow se ejecuta automáticamente cada vez que se realiza un `push` a las ramas configuradas del repositorio.

### 3.2 ¿Qué Hace GitHub Actions?

#### Trigger (Activación)

El workflow se ejecuta automáticamente cuando:

- Se realiza un push a las ramas `main` o `feature-preprocesamiento`
- Se actualiza el repositorio remoto

#### Entorno de Ejecución

GitHub Actions:

- Utiliza una máquina virtual Ubuntu
- Configura automáticamente Python
- Ejecuta el proyecto en un entorno limpio

#### Proceso Automatizado

El workflow realiza automáticamente:

1. Clonado del repositorio
2. Instalación de dependencias
3. Configuración de Python

#### 4. Ejecución del script `preprocesamiento.py`

### Resultado

- Workflow exitoso: el proyecto se ejecuta sin errores
- Workflow fallido: existen errores en el código o dependencias

## 3.3 Beneficios de GitHub Actions

- Detección temprana de errores
- Automatización de pruebas
- Validación continua del proyecto
- Mejora de calidad del código
- Integración continua profesional

```
name: Preprocesamiento CI

on:
 push:
 branches: [main, feature-preprocesamiento]
 pull_request:
 branches: [main]

jobs:
 build:
 runs-on: ubuntu-latest

 steps:
 - uses: actions/checkout@v3

 - name: Set up Python
 uses: actions/setup-python@v4
 with:
 python-version: '3.14'

 - name: Install dependencies
 run: |
 python -m pip install --upgrade pip
 pip install pandas scikit-learn numpy

 - name: Run preprocesamiento.py
 run: python preprocesamiento.py
```

### Explicación del Workflow

- `on`: → Define cuándo se ejecutará el workflow.
- `push`: → Ejecuta el workflow al subir cambios al repositorio.
- `pull_request`: → Ejecuta validaciones automáticas en Pull Requests.
- `runs-on`: → Define el sistema operativo utilizado.
- `actions/checkout`: → Clona automáticamente el repositorio.

- `setup-python`: → Configura el entorno de Python.
- `Install dependencies`: → Instala las librerías necesarias.
- `Run preprocesamiento.py`: → Ejecuta automáticamente el script principal del proyecto.

## 3.2 ¿Qué Hace GitHub Actions?

### Trigger (Activación)

El workflow se ejecuta automáticamente cuando:

- Se realiza un `push` a las ramas `main` o `feature-preprocesamiento`
- Se crea o actualiza una Pull Request

### Entorno de Ejecución

GitHub Actions:

- Utiliza una máquina virtual con Ubuntu (Linux)
- Configura automáticamente Python 3.14

### Proceso Automatizado

El workflow realiza automáticamente las siguientes tareas:

1. Instala o actualiza `pip`
2. Instala las dependencias:
  - `pandas`
  - `scikit-learn`
  - `numpy`
3. Ejecuta el script `preprocesamiento.py`

### Resultado

- Si el script se ejecuta sin errores → Workflow exitoso (verde)
- Si existen errores → Workflow fallido (rojo)

## 3.3 Beneficios de GitHub Actions

### Detección temprana de errores

Permite identificar problemas antes de fusionar cambios al proyecto principal.

### Validación automática

Asegura que el código funcione correctamente en un entorno limpio y automatizado.

### Calidad del código

Ayuda a mantener buenas prácticas y estándares profesionales de desarrollo.

### Ahorro de tiempo

Reduce la necesidad de realizar pruebas manuales repetitivas durante el desarrollo del proyecto.

## 6. Resultados Obtenidos

---

### 6.1 Preprocesamiento Aplicado

Proceso	Resultado
Dataset original	5 filas, 3 columnas
Dataset procesado	4 filas, 6 columnas
Duplicados eliminados	1 fila
Valores nulos imputados	1 valor ( <b>Ingreso</b> )
Variables categóricas	<b>Ciudad</b> → 4 columnas binarias
Normalización	<b>Edad</b> e <b>Ingreso</b> en rango [0,1]

### 6.2 Transformaciones Realizadas

#### Drop Duplicates

Se eliminó la fila con índice **2**, ya que era idéntica a la fila **1**.

#### Fill Na

El valor **NaN** de la columna **Ingreso** (índice **3**) fue reemplazado por la media de la columna:

700.0

#### Get Dummies

La columna categórica **Ciudad** fue transformada en variables binarias:

- **Ciudad\_Cuenca**
- **Ciudad\_Guayaquil**
- **Ciudad\_Loja**
- **Ciudad\_Quito**

#### MinMaxScaler

Las columnas numéricas fueron normalizadas al rango **[0,1]**.

#### Columna Edad

::contentReference[oaicite:0]{index=0}

Transformación aplicada:

```
[20, 25, 30, 35] → [0.0, 0.33, 0.67, 1.0]
```

## Columna Ingreso

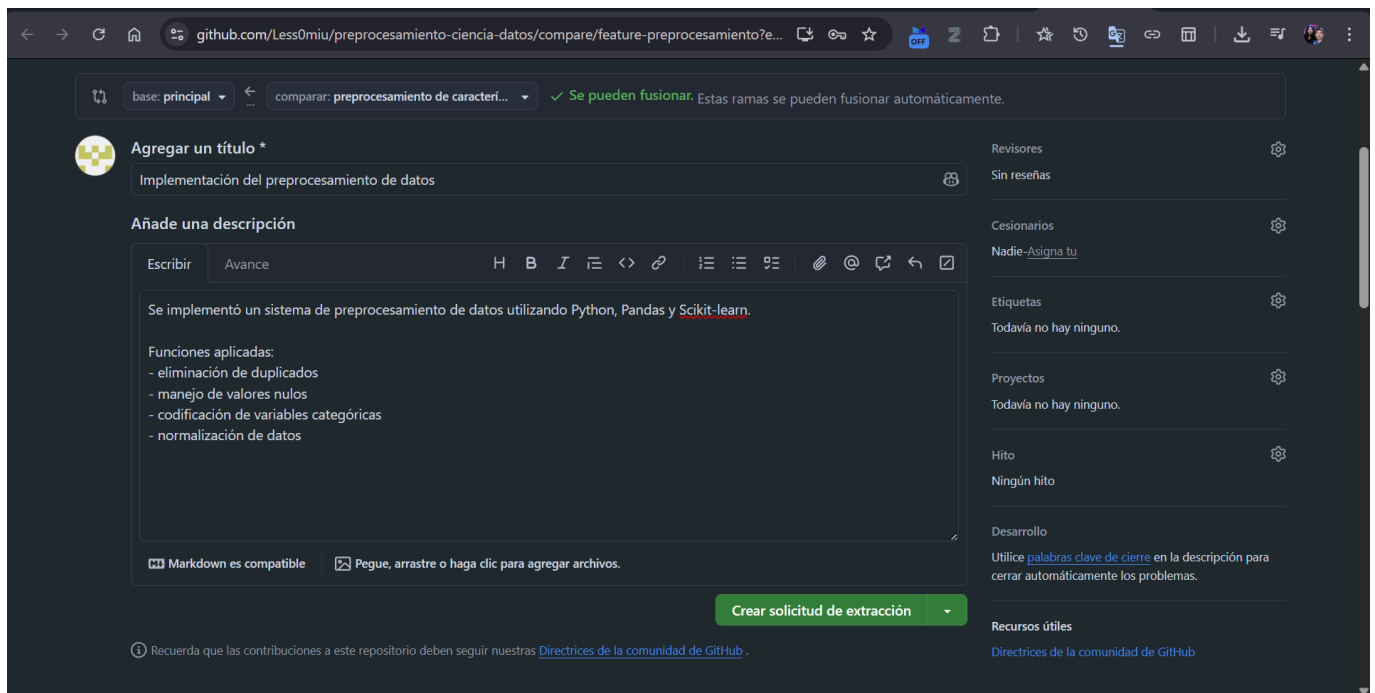
```
::contentReference[oaicite:1]{index=1}
```

Transformación aplicada:

```
[500, 700, 900] → [0.0, 0.5, 1.0]
```

## 7. Capturas de pantalla

### 7.1 Repositorio en GitHub



### 7.2 Ejecución de GitHub Actions

Less0miu / preprocesamiento-ciencia-datos

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

preprocesamiento-ciencia-datos Public

main 2 Branches 0 Tags

Go to file Add file Code

File	Commit Message	Time Ago
.gitignore	Ejecución exitosa del preprocesamiento de datos	2 hours ago
DOCUMENTACION.md	DOCUMENTACION.md	2 hours ago
README.md	Actualización del README	2 hours ago
datos.csv	Se agrega preprocesamiento de datos	4 hours ago
datos_procesados.csv	Se agrega preprocesamiento de datos	4 hours ago
preprocesamiento.py	Ejecución exitosa del preprocesamiento de datos	2 hours ago
preprocesamiento.yml	Agregar workflow de GitHub Actions	2 hours ago

README

About

Proyecto de preprocesamiento de datos con Git y GitHub

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published [Create a new release](#)

Packages

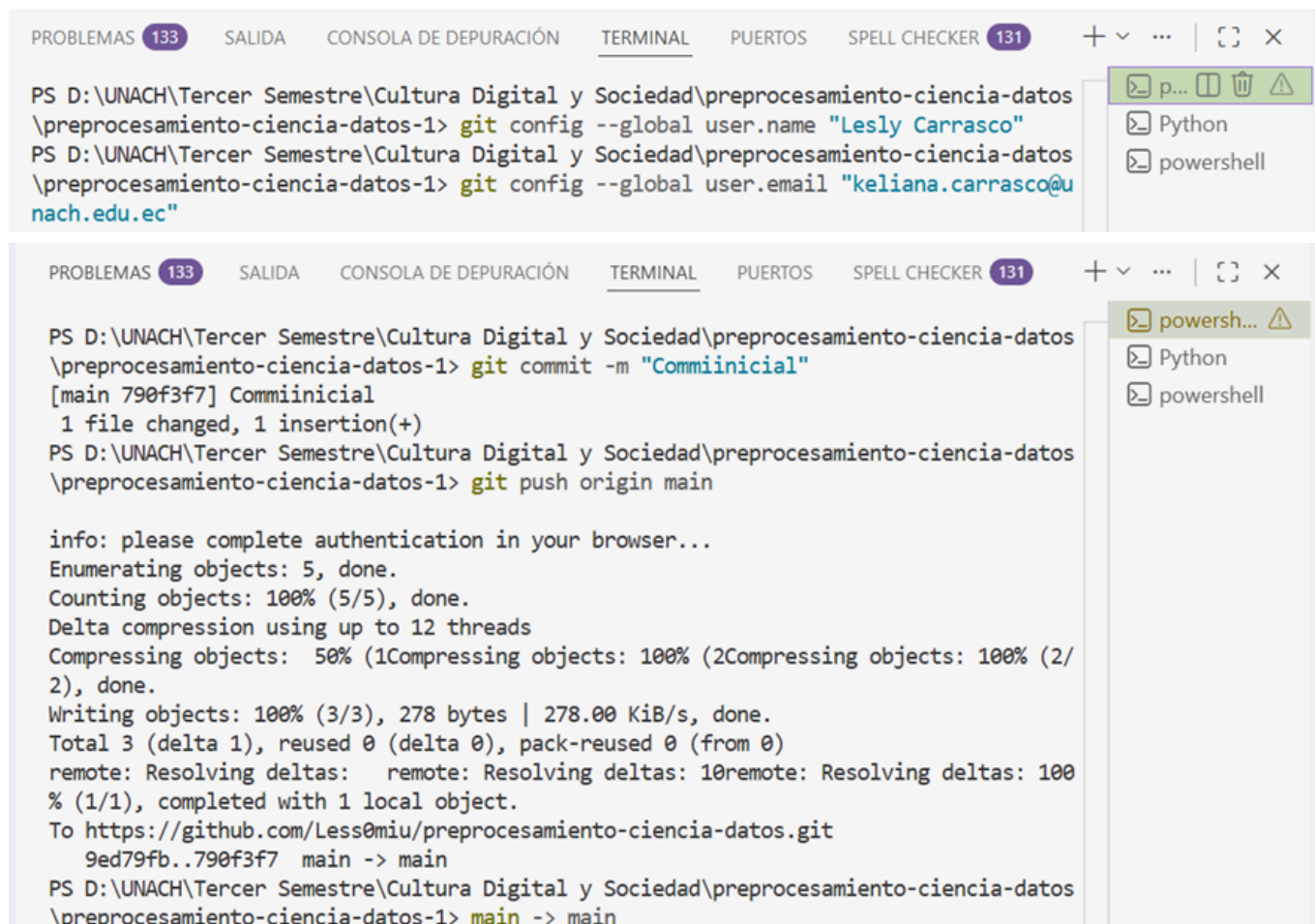
No packages published [Publish your first package](#)

### 7.3 Resultado del preprocesamiento

```
PS D:\UNACH\Tercer Semestre\Cultura Digital y Sociedad\preprocesamiento-ciencia-datos\preprocesamiento-ciencia-datos-1> python preprocesamiento.py
=== CONJUNTO DE DATOS ORIGINAL ===
 Edad Ingreso Ciudad
0 20 500.0 Quito
1 25 700.0 Guayaquil
2 25 700.0 Guayaquil
3 30 NaN Cuenca
4 35 900.0 Loja
=== CONJUNTO DE DATOS PROCESADO ===
 Edad Ingreso Ciudad_Cuenca Ciudad_Guayaquil Ciudad_Loja Ciudad_Quito
0 0.000000 0.0 0.0 0.0 0.0 1.0
1 0.333333 0.5 0.0 1.0 0.0 0.0
3 0.666667 0.5 1.0 0.0 0.0 0.0
4 1.000000 1.0 0.0 0.0 1.0 0.0
```

### 7.4 Adicionales





The image shows two screenshots of a VS Code terminal window. The top screenshot shows the terminal with the following commands and output:

```
PS D:\UNACH\Tercer Semestre\Cultura Digital y Sociedad\preprocesamiento-ciencia-datos
\preprocesamiento-ciencia-datos-1> git config --global user.name "Lesly Carrasco"
PS D:\UNACH\Tercer Semestre\Cultura Digital y Sociedad\preprocesamiento-ciencia-datos
\preprocesamiento-ciencia-datos-1> git config --global user.email "keliana.carrasco@u
nach.edu.ec"
```

The bottom screenshot shows the terminal with the following commands and output:

```
PS D:\UNACH\Tercer Semestre\Cultura Digital y Sociedad\preprocesamiento-ciencia-datos
\preprocesamiento-ciencia-datos-1> git commit -m "Commiinicial"
[main 790f3f7] Commiinicial
1 file changed, 1 insertion(+)
PS D:\UNACH\Tercer Semestre\Cultura Digital y Sociedad\preprocesamiento-ciencia-datos
\preprocesamiento-ciencia-datos-1> git push origin main

info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 50% (1Compressing objects: 100% (2Compressing objects: 100% (2/
2), done.
Writing objects: 100% (3/3), 278 bytes | 278.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: remote: Resolving deltas: 10remote: Resolving deltas: 100
% (1/1), completed with 1 local object.
To https://github.com/Less0miu/preprocesamiento-ciencia-datos.git
9ed79fb..790f3f7 main -> main
PS D:\UNACH\Tercer Semestre\Cultura Digital y Sociedad\preprocesamiento-ciencia-datos
\preprocesamiento-ciencia-datos-1> main -> main
```

## 8. Conclusiones

- Se logró implementar un pipeline básico de preprocesamiento de datos.
- Se aplicaron buenas prácticas de control de versiones con Git y GitHub.
- Se automatizó la validación del código utilizando GitHub Actions.
- El proyecto quedó preparado para futuras mejoras y ampliaciones.
- Se fortalecieron conocimientos sobre limpieza, transformación y automatización de datos en proyectos de ciencia de datos.